Seminario de Arquitecturas de Software

Un Análisis Comparativo de Estilos Basados en Capas y Dominio

Equipo: $RAM + L^2$

Integrante	Grupo	Pregunta(s)
Raúl R. Espinosa Poma	C312	5, 6
Ailema Matos Rodríguez	C312	3
Meylí Jiménez Velazquez	C312	2
Lianet Tamarit Tejas	C311	4
Luis D. Friol Rodríguez	C312	1

Repositorio: `https://github.com/Charge1500/nutritional_management_system`

Índice

Introducción **3**

1. Análisis de Trade-offs: Arquitectura N-Capas vs. Clean/Onion/Hexagonal **3**
- 1.1. Problema de la arquitectura n-capas tradicional . . . 3
- 1.2. Regla de dependencia . . . 3
- 1.3. Arquitectura hexagonal . . . 4
- 1.4. Pruebas (testing) . . . 4
- 1.5. Migración a MongoDB . . . 4

2. Comparación de Estilos: la Simetría del Hexágono vs. la Jerarquía de las Capas **4**
- 2.1. ¿Qué problema conceptual resuelve la simetría de la representación hexagonal que una representación de capas apiladas (como la de Onion o la N-Capas tradicional) no logra transmitir? . . . 4
- 2.2. ¿Cómo se traduce el concepto de Puerto y Adaptador en la práctica de definir interfaces en las capas internas de la Arquitectura Onion? . . . 6
- 2.3. ¿Por qué Cockburn afirma que el hexágono no tiene seis lados por una razón numérica, sino para eliminar la asimetría izquierda-derecha o arriba-abajo? 7

3. Aplicación de DDD y Bounded Context en un Dominio Complejo **7**
- 3.1. ¿Cómo dividiría este sistema en varios contextos delimitados? . . . 7
- 3.2. ¿Cómo evitaría que el modelo de Póliza se contamine con detalles de siniestros o facturación? . . . 9
- 3.3. ¿Cómo se comunicarían estos contextos de manera desacoplada? . . . 9
- 3.4. ¿Cómo se relaciona esta solución con una Arquitectura Orientada al Dominio y el Adaptador de Servicio? . . . 10

4. Evaluación de la Inversión de Dependencias en la Práctica **12**
4.1. Violación de los principios arquitectónicos y de la Regla de Dependencia . 12
4.2. Rediseño mediante el Principio de Inversión de Dependencias 13
4.3. Impacto en la testabilidad, la mantenibilidad y la flexibilidad 14
4.4. Pruebas unitarias de la solución propuesta 14

5. Análisis de un Problema Real: El Kitty Problem y el Principio de Abierto/Cerrado **15**
5.1. Descripción del problema 15
5.2. Por qué la división por verbos llevó a esto 16
5.3. Cómo diseñarlo para que el cambio afecte a un solo componente 17
5.4. El Principio de Abierto/Cerrado 18
5.5. Papel de los adaptadores en la extensión 19

6. Evaluación de la Arquitectura N-Capas frente a Requisitos de Atributos de Calidad **19**
6.1. Análisis de la afirmación: capas abiertas y fast-lane reader 19
6.2. El anti-patrón sinkhole y la regla del 80/20 20
6.3. Por qué Hexagonal y Limpia son más adecuadas para alta escalabilidad y agilidad 21

Conclusiones **21**

Bibliografía **22**

Declaración de Uso de la Inteligencia Artificial **23**

Introducción

El diseño de la arquitectura de software es una herramienta de pensamiento: implica decidir cómo organizar las responsabilidades de un sistema y cómo establecer las dependencias entre sus componentes. En este contexto, este seminario propone comparar los estilos arquitectónicos: N-Capas tradicional, Hexagonal, Onion, Clean Architecture y sus principios de organización y aislamiento del negocio. Más allá de sus representaciones visuales, el interés de esta comparación consiste en comprender cómo cada enfoque aborda la separación entre la lógica de negocio y los mecanismos técnicos necesarios para interactuar con usuarios, bases de datos y servicios externos. En particular, Cockburn y Palermo coinciden en la importancia de externalizar la infraestructura para evitar que sus detalles condicionen el núcleo de la aplicación. (Cockburn, 2005; Palermo, 2008).

Este seminario se centrará en responder las preguntas propuestas, las cuales examinan la relación entre la forma de representar una arquitectura y los conceptos que esa representación permite comprender. A partir de esta comparación, el seminario conecta los principios arquitectónicos con una solución concreta basada en el Principio de Inversión de Dependencias. Finalmente, con las pruebas unitarias se incorpora una comprobación acotada de esa solución. Así, en este seminario se desarrolla un recorrido que va desde la interpretación conceptual de las arquitecturas hasta la aplicación de sus principios y la evaluación de comportamientos específicos, distinguiendo la independencia del núcleo de la verificación integral de los servicios externos.

Pregunta 1. Análisis de Trade-offs: Arquitectura N-Capas vs. Clean/Onion/Hexagonal

1.1. Problema de la arquitectura n-capas tradicional

En la arquitectura de n-capas tradicional (flujo de control: Presentación → Negocio → Datos), se obliga a la lógica de negocio a conocer y depender directamente de detalles técnicos de infraestructura, al depender la capa de negocio directamente de la capa de datos.

- Esta dependencia viola el principio de inversión de dependencias, debido a que los módulos de alto nivel dependen de los de bajo nivel.
- Como consecuencia, cualquier cambio en la base de datos obliga a modificar la lógica de negocio, y las pruebas unitarias requieren una base de datos real o mocks complejos.

1.2. Regla de dependencia

La regla de dependencia plantea que las dependencias del código fuente solo pueden apuntar hacia las capas de mayor nivel.

- La capa de negocio define una interfaz que expresa qué necesita para persistir datos, y la implementación concreta se coloca en la capa externa de infraestructura, invirtiendo la dependencia.
- La infraestructura depende de la abstracción definida en el negocio, y no al revés.

1.3. Arquitectura hexagonal

La arquitectura hexagonal propone aislar el núcleo de negocio mediante interfaces definidas como puertos e implementaciones concretas definidas como adaptadores.

- El núcleo define un puerto de transferencia, y fuera del hexágono se crean adaptadores que implementan ese mismo puerto.
- El núcleo no necesita saber qué adaptador está conectado; solo conoce la interfaz.
- Esto permite que la base de datos, la UI o cualquier servicio externo puedan ser intercambiados sin tocar el núcleo.

1.4. Pruebas (testing)

Arquitectura limpia. Para las pruebas se puede crear un fake o un mock, de modo que los casos de uso se ejecuten sin usar la base de datos real ni librerías de persistencia, y de forma sencilla.

Arquitectura hexagonal. El núcleo expone un puerto y se puede implementar un adaptador de prueba (fake o mock) que simule la persistencia. El núcleo se prueba sin infraestructura, y la simetría del hexágono permite sustituir los componentes externos por "dobles de prueba", reduciendo la fragilidad y acelerando el ritmo de desarrollo.

1.5. Migración a MongoDB

Arquitectura limpia. La capa de infraestructura implementa la interfaz definida en el núcleo. Para migrar a MongoDB se crea una nueva implementación en la infraestructura. Los casos de uso y las entidades no se modifican; solo cambia la inyección de dependencias para que use la nueva implementación. El cambio queda confinado a la capa externa.

Arquitectura hexagonal. Se crea un nuevo adaptador que implemente el puerto que se comunica con el núcleo. El núcleo no se entera del cambio; solo se reemplaza el adaptador en el exterior del hexágono. La simetría garantiza que el cambio no afecte a los demás adaptadores ni al núcleo.

Pregunta 2. Comparación de Estilos: la Simetría del Hexágono vs. la Jerarquía de las Capas

2.1. ¿Qué problema conceptual resuelve la simetría de la representación hexagonal que una representación de capas apiladas (como la de Onion o la N-Capas tradicional) no logra transmitir?

La representación hexagonal resuelve un problema de perspectiva ya que permite comprender que la interfaz de usuario, la base de datos y los demás sistemas externos son elementos que deben estar posicionados fuera de la aplicación (Cockburn, 2018, 308), y que el obstáculo estructural principal es lograr separar la lógica de negocio de las interacciones

con esos elementos. Su simetría destaca que, aunque dichos elementos cumplen funciones distintas, todos deben relacionarse con la aplicación mediante límites explícitos que protejan al interior (Cockburn, 2005, 2).

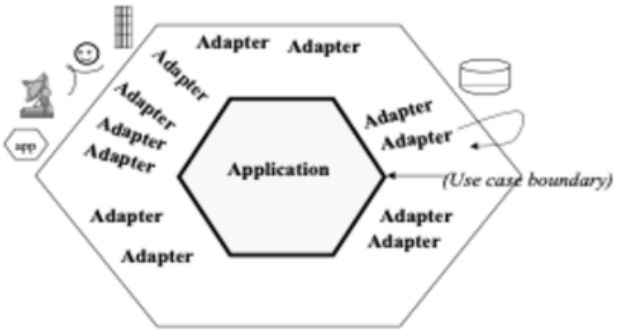


Figura 1: Arquitectura hexagonal (Cockburn, 2005, 2)

Es importante tener en cuenta que al Onion ser una arquitectura de capas concéntricas con un núcleo interno y una infraestructura externa (Palermo, 2008, 2), el problema conceptual que le soluciona la representación hexagonal no es en introducirle una separación que no tenga (como a la N-Capas), sino en destacar la similitud de las interacciones externas y la presencia de diversos puertos alrededor de la aplicación.

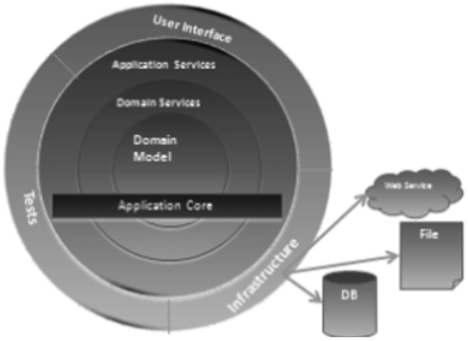


Figura 2: Arquitectura Onion (Palermo, 2008, 2)

El principal problema que la arquitectura hexagonal resuelve es evitar la confusión que se crea con la posición de los elementos y su interdependencia. En una representación tradicional de tres capas (presentación, negocio y datos), la aplicación suele verse como una secuencia vertical: el usuario interactúa con la presentación, ésta utiliza la lógica de negocio y esta última usa el acceso a datos. La dificultad conceptual aparece cuando esa separación hace parecer que la interfaz de usuario y el almacenamiento pertenecen a categorías arquitectónicas diferentes (Cockburn, 2005, 11): la primera estaría "por encima" de la aplicación y la segunda sería su "base". La arquitectura hexagonal reconoce que ambas están fuera del límite de la aplicación. Para ser más específicos, esta arquitectura considera que a pesar de que la interfaz gráfica y una base de datos no cumplen las mismas funciones, ambas como tecnologías externas no deberían determinar la lógica de negocio.

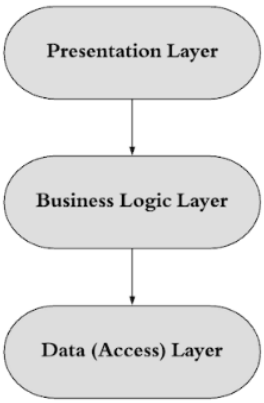


Figura 3: Arquitectura de 3 Capas (Yaghini, 2025)

Por tanto, el problema recae en la idea de que la separación gráfica garantiza la independencia y no en la existencia de capas en sí. La representación hexagonal muestra que añadir una nueva capa con el compromiso de no introducirle lógica de negocio, no resuelve el problema si no existe una manera de detectar cuándo se viola esa separación. También identifica un problema equivalente: la lógica de la aplicación queda ligada a una base de datos o a otro servicio externo. Si ese servicio deja de estar disponible, se modifica considerablemente o se sustituye, el trabajo de los programadores puede quedar bloqueado porque está atado a la presencia de la base de datos. La causa de ambos problemas es no haber protegido adecuadamente el límite de la aplicación (Cockburn, 2005, 3).

La solución que la simetría aporta consiste en aislar sus detalles al sustituir el eje visual dominante de las capas apiladas por una frontera: dentro la aplicación y afuera las entidades con las que se comunica. La simetría también permite reconocer que la sustitución de tecnologías debe ser posible tanto en las entradas como en las salidas. Por ejemplo, desde la perspectiva de la aplicación, si la base de datos es cambiada de SQL a un archivo o a cualquier otro tipo de base de datos, la interacción a través de la API (*application programming interface*) no debería cambiar (Cockburn, 2005, 3). Por tanto, la interfaz gráfica deja de ser la única puerta de entrada y la base de datos deja de ser una plataforma obligatoria para ejecutar todas las pruebas. La aplicación puede ser conducida por pruebas automatizadas (que también permiten detectar filtraciones de lógica hacia la presentación) y se puede usar sustitutos de sus servicios externos para correrlos de manera aislada (Cockburn, 2005, 4). Así se puede ejecutar y probar el comportamiento de la aplicación sin la interfaz gráfica ni la base de datos real. La asimetría relevante no es la que existe entre izquierda y derecha, sino entre el interior y el exterior (Cockburn, 2005, 8).

2.2. ¿Cómo se traduce el concepto de Puerto y Adaptador en la práctica de definir interfaces en las capas internas de la Arquitectura Onion?

Primeramente, según Cockburn un puerto es una conversación con propósito cuyo protocolo toma la forma de una API, mientras que un adaptador es el componente que convierte esa definición de API en las señales que necesita cada dispositivo externo, y viceversa (Cockburn, 2005, 3). En el caso de un puerto de salida una realización consiste en definir una interfaz en una capa interna del núcleo y colocar su implementación en infraestructura; esta es una correspondencia práctica entre ambas arquitecturas. Debe

quedar claro que en la práctica definir interfaces en las capas internas significa que el núcleo declara puertos (interfaces) y la infraestructura externa proporciona adaptadores (implementaciones).

Además, Cockburn distingue puertos y actores primarios, que conducen la aplicación (sacándola de su estado de reposo) y secundarios, a los que la aplicación conduce para obtener respuestas o notificaciones (Cockburn, 2005, 9). Llevado a la práctica de Onion, un adaptador entrante invoca las operaciones que el núcleo expone, mientras que un adaptador saliente implementa una interfaz definida internamente. Para cada puerto existen típicamente varios adaptadores: interfaz gráfica, herramienta de pruebas automatizadas como FIT o Fitnesse, controlador por lotes, interfaz http, o interfaz directa entre programas; y en el lado de datos, un adaptador SQL, uno de archivo plano y una base de datos "mock" en memoria que no depende de la presencia de la base de datos real (Cockburn, 2005, 3). Aquí se ve el mayor beneficio de una implementación de puertos y adaptadores: otorga la capacidad de ejecutar la aplicación de forma aislada, aportando la flexibilidad y testabilidad que busca el patrón.

2.3. ¿Por qué Cockburn afirma que el hexágono no tiene seis lados por una razón numérica, sino para eliminar la asimetría izquierda-derecha o arriba-abajo?

Cockburn dice textualmente que el hexágono no es un hexágono porque el número seis sea importante, sino para permitir que quienes hacen el dibujo tengan espacio para insertar puertos y adaptadores según lo necesiten, sin estar limitados por un dibujo unidimensional de capas. El término arquitectura hexagonal proviene de ese efecto visual (Cockburn, 2005, 4).

Con el dibujo de capas muchas aplicaciones tienen solo dos puertos: el diálogo con el usuario y el diálogo con la base de datos, lo que les da una apariencia asimétrica y hace que parezca natural construir la aplicación como una arquitectura apilada de tres a cinco capas. Cockburn identifica dos problemas en esos dibujos. El primero: las personas tienden a no tomarse en serio las "líneas" del dibujo de capas, dejando que la lógica de la aplicación se filtre a través de los límites. El segundo: puede haber más de dos puertos, de modo que la arquitectura no cabrá en el dibujo unidimensional (Cockburn, 2005, 4).

La arquitectura de hexágono resuelve estos problemas al eliminar cualquier asimetría izquierda-derecha o arriba-abajo, y permite ver que la aplicación se comunica a través de puertos con agentes externos. La asimetría notable no es la que existe entre los lados izquierdo y derecho de la aplicación, sino entre el interior y el exterior, lo cual además permite destacar visualmente la presencia de un número definido de puertos distintos (Cockburn, 2005, 4).

Pregunta 3. Aplicación de DDD y Bounded Context en un Dominio Complejo

3.1. ¿Cómo dividiría este sistema en varios contextos delimitados?

El enunciado muestra que una misma palabra, póliza, significa cosas distintas según quién la use: para el actuario es un contrato con cláusulas, para facturación es una fuente

de ingresos y para el ajustador es un número de referencia. Esto indica que no existe un único modelo válido para todo el sistema, y que lo adecuado es dividirlo en varios modelos con fronteras claras, que es la idea central del *bounded context* de DDD. Evans lo define como el contexto explícito dentro del cual se aplica un modelo: sus límites se establecen en términos de la organización de los equipos, del uso en partes concretas de la aplicación y de manifestaciones físicas como las bases de código y los esquemas de bases de datos, y dentro de ellos el modelo se mantiene estrictamente coherente, sin dejarse arrastrar por problemas externos (Evans, 2003, cap. 14). La guía de arquitectura N-Capas orientada al dominio explica el mismo concepto: a medida que los modelos crecen resulta muy difícil mantener su coherencia, por lo que se separan en varios modelos más pequeños, cada uno válido dentro del contexto donde está definido; los contextos son particiones del modelo destinadas a mantener la coherencia, no una simple partición funcional del mismo (De la Torre Llorente et al., 2010, secc. 2.7).

Richards y Ford explican que este concepto, tomado de DDD, representa un estilo de desacoplamiento: dentro de un contexto, el código y los esquemas de datos están acoplados entre sí, pero nunca con elementos de otro contexto, como la base de datos o la definición de una clase ajena. En un monolito tradicional, en cambio, se suelen compartir conceptos como clientes o pagos mediante clases reutilizables y bases de datos enlazadas, y la contrapartida de esa reutilización es el acoplamiento; por eso, cuando se busca un desacoplamiento alto, se prefiere la duplicación a la reutilización (Richards y Ford, 2020, cap. 17).

Aplicado al sistema de seguros, se proponen cuatro contextos:

- **Suscripción y Gestión de Pólizas:** crea, emite y administra el contrato de seguro. Aquí la póliza es un contrato con cláusulas y coberturas.
- **Facturación de Primas:** administra los cobros. Aquí la póliza es una fuente de ingresos con un plan de pagos, y se ignoran las cláusulas técnicas.
- **Gestión de Siniestros:** administra la reclamación y la indemnización tras un accidente. Aquí la póliza es un registro de vigencia y de límites de cobertura.
- **Evaluación en Campo:** corresponde a la aplicación del ajustador. Aquí la póliza se reduce a una referencia que permite comprobar rápidamente que existe cobertura.

Esta división no sigue pasos funcionales (cobrar, evaluar, emitir), sino el significado que el negocio da a sus términos. Evans propone representar las relaciones entre contextos en un mapa de contexto (Evans, 2003, cap. 14), y la guía N-Capas recomienda definirlo a la vez que los contextos, para que queden claras sus relaciones (De la Torre Llorente et al., 2010, secc. 2.7). En este caso, Suscripción es el contexto del que dependen los demás (*upstream*), ya que es quien crea y modifica la póliza; Facturación y Siniestros consumen información de ella (*downstream*), y Evaluación en Campo depende a su vez de Siniestros. Sobre estas relaciones se proponen los patrones de Evans: Suscripción expone un *Open Host Service* con un *Published Language*, es decir, un servicio y un formato de mensajes estables y documentados que los demás consumen sin conocer su modelo interno (Evans, 2003, cap. 14); y cada contexto consumidor se protege con una capa anticorrupción (*ACL*) que traduce ese formato a su propio lenguaje (Evans, 2003, cap. 14). La guía N-Capas describe esta misma capa en su apartado *Anti-corruption Layer* (De la Torre Llorente et al., 2010, secc. 2.8.4).

3.2. ¿Cómo evitaría que el modelo de Póliza se contamine con detalles de siniestros o facturación?

La contaminación ocurre cuando la entidad Póliza acumula campos de cobro, de reclamaciones y de inspección que no le corresponden. La primera medida para evitarlo es que cada contexto tenga su propio modelo: la entidad Póliza, con sus cláusulas y coberturas, existe únicamente en Suscripción, y los demás contextos definen su propia representación con los datos que necesitan, por ejemplo *PlanDePagos* en Facturación, *CoberturaVigente* en Siniestros y *ReferenciaDePóliza* en Evaluación en Campo. Además, Richards y Ford señalan que el concepto de contexto delimitado exige aislamiento de datos: se evitan los esquemas y las bases de datos compartidos como puntos de integración (Richards y Ford, 2020, cap. 17), idea que se traslada a los contextos: ninguno comparte esquema con los demás. Por la misma razón se descarta el patrón *Shared Kernel*, en el que dos contextos comparten ciertos objetos y cualquier modificación de ellos requiere la aprobación de los equipos de todos los contextos implicados (De la Torre Llorente et al., 2010, secc. 2.8.1; Evans, 2003, cap. 14). Compartir la entidad Póliza reintroduciría justamente el acoplamiento que se quiere evitar.

La segunda medida es que un contexto no guarde la póliza completa de otro, sino su identificador y los pocos datos que le interesan. Richards y Ford indican que, al desaparecer la fuente única de verdad, hay que decidir entre identificar un dominio como fuente de cierto dato y coordinarse con él para obtener los valores, o usar replicación o caché para distribuir la información (Richards y Ford, 2020, cap. 17). Así, un cambio en las cláusulas del contrato no afecta a Facturación ni a Siniestros.

Como tercera medida, la traducción en la frontera mediante adaptadores de infraestructura evita que los conceptos externos se filtren al modelo del dominio (aspecto que se desarrolla en detalle en la sección 3.4).

Por último, la arquitectura Onion refuerza estas medidas dentro de cada contexto: el modelo de dominio ocupa el centro y las capas externas dependen de las internas, nunca al revés (Palermo, 2008a). De este modo, el núcleo del contexto de Suscripción no depende de la infraestructura ni de las tecnologías con las que se comunican los demás contextos. Que los conceptos de siniestros o de facturación no entren al modelo de Póliza lo garantizan las tres medidas anteriores (modelo propio, datos mínimos y traducción en la frontera), no la dirección de las dependencias por sí sola.

3.3. ¿Cómo se comunicarían estos contextos de manera desacoplada?

La comunicación entre contextos se plantea de modo que ninguno necesite conocer los detalles internos del otro. Para los hechos relevantes del negocio se usan eventos de integración asíncronos: cuando ocurre algo, el contexto de origen publica un evento sin saber quién lo consume. Por ejemplo, Suscripción publica `PolizaEmitida` o `PolizaCancelada`; Facturación reacciona creando o cancelando el plan de pagos, y Siniestros actualiza su copia local de la vigencia y de los límites. Richards y Ford describen este estilo, llamado coreografía: no existe un coordinador central, lo que respeta la filosofía del contexto delimitado, y resulta natural implementar eventos desacoplados entre los servicios (Richards y Ford, 2020, cap. 17). Los eventos forman parte del *Published Language* de Suscripción: son mensajes con un formato estable y versionado, distinto del modelo interno de la entidad Póliza (Evans, 2003, cap. 14).

Esta elección tiene contrapartidas. En la coreografía se preserva la filosofía de desacoplamiento, pero problemas como el manejo de errores y la coordinación se vuelven más complejos; la alternativa es la orquestación, en la que un mediador local centraliza la coordinación de un proceso a costa de acoplar los servicios que coordina (Richards y Ford, 2020, cap. 17). Si algún proceso entre contextos (por ejemplo, un pago de indemnización que involucre a Siniestros y Facturación) exigiera un control estricto de sus pasos, podría coordinarse con un mediador local solo para ese caso, sin cambiar la comunicación por eventos del resto. Los mismos autores aconsejan evitar las transacciones entre servicios y, si fueran inevitables, recurrir al patrón saga, con serios compromisos de complejidad (Richards y Ford, 2020, cap. 17). Además, la información entre contextos tiene consistencia eventual y no inmediata.

Para lo que necesita respuesta inmediata, como que el ajustador compruebe en el lugar del accidente si existe cobertura, el contexto de Siniestros expone una consulta cuyo resultado es un objeto de transferencia de datos (DTO) mínimo, con la referencia, la vigencia y el límite. De esta forma no se expone la entidad interna del dominio. En ambos casos lo que se publica es un contrato estable y más pequeño que el modelo interno: si el modelo de Suscripción cambia, mientras el contrato se mantenga, los demás contextos no se ven afectados.

3.4. ¿Cómo se relaciona esta solución con una Arquitectura Orientada al Dominio y el Adaptador de Servicio?

DDD indica qué se debe proteger, que es el modelo de cada contexto, y las arquitecturas Onion y Limpia indican cómo protegerlo estructuralmente. Cada contexto delimitado se implementa como una aplicación con su propio núcleo de dominio. En la Onion de Palermo, ese núcleo tiene en el centro el modelo de dominio, a su alrededor los servicios de dominio y de aplicación, y en el exterior la infraestructura y la interfaz de usuario, con las interfaces (por ejemplo, las de repositorio) definidas en las capas internas (Palermo, 2008a). En la Arquitectura Clean, la Regla de Dependencia exige que el código fuente apunte siempre hacia las políticas de mayor nivel (Martin, 2017, p. 203). Cuando el núcleo de un contexto necesita algo de otro, no depende de él directamente: define una interfaz con lo que necesita, que es el puerto, y la infraestructura aporta la implementación, que es el adaptador. Para Cockburn, un puerto es una conversación con propósito, y a cada puerto se pueden conectar varios adaptadores (Cockburn, 2005, sección “Nature of the Solution”).

La Figura 4 muestra esta regla: cada círculo es una capa, y las dependencias del código fuente apuntan siempre hacia el centro, donde están las reglas de negocio. Las interfaces, controladores y detalles como la base de datos o la interfaz de usuario quedan en los círculos exteriores.

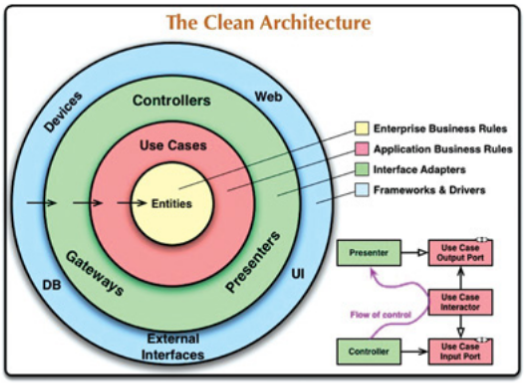


Figure 22.1 The clean architecture

THE DEPENDENCY RULE

Figura 4: (Martin, 2017, Fig. 22.1).

La Guía de Arquitectura N-Capas orientada al Dominio recoge esta misma idea en la capa de infraestructura, donde sitúa a los Agentes de Servicios Externos, que se usan cuando un componente de negocio necesita funcionalidad proporcionada por servicios externos o remotos (De la Torre Llorente et al., 2010, secc. 2.1.4 y 5.14). Es lo que el enunciado llama Adaptador de Servicio: el componente que encapsula los detalles de la comunicación con el servicio ajeno y traduce entre su lenguaje y el del dominio. De este modo, el núcleo del contexto nunca conoce el protocolo ni el modelo del otro contexto.

Cuando esa comunicación es entre dos contextos delimitados, el Adaptador de Servicio es la pieza que implementa la capa anticorrupción vista en la pregunta anterior. Según la guía, esta capa se compone de tres elementos: una fachada que simplifica la comunicación con el otro contexto y se define en términos de su modelo, un adaptador que ajusta la interfaz del otro contexto a la que espera el contexto propio, y un traductor que mapea los elementos de un modelo al otro (De la Torre Llorente et al., 2010, secc. 2.8.4). Es un adaptador saliente, que implementa un puerto definido en el interior: se distinguen los adaptadores primarios, que conducen la aplicación, de los secundarios, a los que la aplicación conduce (Cockburn, 2005, sección "Application Notes").

El Listado 1 muestra cómo el contexto de Siniestros comprueba la vigencia de una póliza sin depender del contexto de Suscripción. En él, `IPolizaApiClient` hace de fachada, porque habla en términos del modelo del otro contexto (`PolizaDto`); `PolizaServiceAdapter` es el adaptador que cumple el puerto que espera Siniestros; y la conversión a `CoberturaVigente` es el traductor, que podría extraerse a una clase propia si crece.

```
// NÚCLEO del contexto Siniestros: puerto definido por lo que necesita el dominio
public interface IConsultaVigenciaPoliza {
    CoberturaVigente ObtenerVigencia(string referenciaPoliza, DateTime fecha);
}

// INFRAESTRUCTURA: adaptador de servicio (traduce entre contextos)
public class PolizaServiceAdapter : IConsultaVigenciaPoliza {
    private readonly IPolizaApiClient _cliente;   // cliente HTTP del contexto
    Suscripción
    public PolizaServiceAdapter(IPolizaApiClient cliente) { _cliente = cliente; }

    public CoberturaVigente ObtenerVigencia(string referenciaPoliza, DateTime fecha) {
        PolizaDto dto = _cliente.Obtener(referenciaPoliza);  // modelo del otro
```

```
    contexto
        // traducción al lenguaje de Siniestros
        bool vigente = fecha >= dto.VigenteDesde && fecha <= dto.VigenteHasta;
        return new CoberturaVigente(dto.Numero, vigente, dto.LimiteMaximo);
    }
}
```

Listing 1: Puerto en el núcleo de Siniestros y adaptador de servicio en infraestructura.

El Listado 1 corresponde a la variante síncrona de la comunicación (consulta). Tiene la desventaja de que, si Suscripción no está disponible, Siniestros no puede comprobar la vigencia en ese momento. La variante por eventos descrita en el apartado anterior evita esa dependencia en tiempo de ejecución: un adaptador entrante en la infraestructura de Siniestros recibe `PolizaEmitida` y `PolizaCancelada`, los traduce y actualiza la copia local de *CoberturaVigente*, y el adaptador de consulta pasa a leer de esa copia. En ambos casos el puerto del núcleo es el mismo, por lo que el dominio de Siniestros no se entera de cuál de las dos variantes se utiliza.

Este diseño tiene las mismas ventajas vistas en la Pregunta 4. El núcleo de Siniestros puede probarse con una implementación simulada de `IConsultaVigenciaPoliza`, sin llamar a Suscripción; la guía N-Capas señala que los agentes de servicios externos son componentes especialmente adecuados para desacoplarse mediante inversión de control y simularse con *fakes* en pruebas (De la Torre Llorente et al., 2010, secc. 5.14). Cockburn añade que el beneficio último de puertos y adaptadores es poder ejecutar la aplicación en modo completamente aislado, con adaptadores de prueba o simulados en lugar de los reales (Cockburn, 2005, sección “Application Notes”). Y si Suscripción cambia su API, el cambio se limita al adaptador y no alcanza al dominio de Siniestros.

Pregunta 4. Evaluación de la Inversión de Dependencias en la Práctica

4.1. Violación de los principios arquitectónicos y de la Regla de Dependencia

En la implementación descrita, la clase `OrderProcessor`, responsable de la lógica de negocio relacionada con el procesamiento de pedidos, instancia y utiliza directamente la clase concreta `AddressValidatorService`. Esta última depende, a su vez, de una biblioteca HTTP específica para comunicarse con el servicio externo de validación de direcciones.

Esta estructura genera un acoplamiento entre la lógica de negocio y los detalles de infraestructura. El problema fundamental consiste en que un componente de alto nivel, que contiene políticas de negocio, depende directamente de una implementación concreta de bajo nivel, encargada de la comunicación con un sistema externo.

Esta dependencia viola el Principio de Inversión de Dependencias (DIP), explicado en el capítulo 11 por Martin (2017). Asimismo, incumple la Regla de Dependencia de Clean Architecture, presentada en el capítulo 22. Según esta última, las dependencias del código fuente deben dirigirse hacia las políticas de mayor nivel, de modo que el núcleo de la aplicación no dependa de los detalles tecnológicos externos (Martin, 2017).

La misma situación contradice el objetivo de la Arquitectura Hexagonal, que busca aislar la aplicación de las tecnologías externas mediante puertos y adaptadores (Cockburn,

2005), y el de la Arquitectura Cebolla, que establece que las capas externas deben depender de las internas, y no al contrario (Palermo, 2008).

Por tanto, el problema no radica en utilizar una biblioteca HTTP, sino en permitir que la lógica de negocio conozca directamente la implementación concreta que utiliza dicha biblioteca. Esto dificulta las pruebas aisladas y aumenta el impacto de los cambios tecnológicos sobre el núcleo de la aplicación.

4.2. Rediseño mediante el Principio de Inversión de Dependencias

Para corregir el problema, se propone introducir una abstracción en el núcleo de la aplicación y trasladar la implementación concreta de la comunicación externa a la capa de infraestructura. Este diseño se basa en el principio de que los módulos de alto nivel y los de bajo nivel deben depender de abstracciones, en lugar de que los primeros dependan directamente de los segundos (Martin, 2017).

La solución se estructura en dos componentes principales:

1. **Puerto en el núcleo de la aplicación:** se define una interfaz denominada `IAddressValidator` (código 1), que establece el contrato necesario para validar una dirección. Esta interfaz expresa lo que necesita la aplicación, sin especificar cómo se realiza la validación ni qué tecnología se utiliza. La clase `OrderProcessor` (código 2) recibe una implementación de esta interfaz mediante inyección de dependencias y trabaja únicamente con el contrato definido.

2. **Adaptador en la capa de infraestructura:** se implementa una clase concreta, por ejemplo, `HttpAddressValidatorAdapter` (código 3), que realiza la comunicación con el servicio externo utilizando la biblioteca HTTP seleccionada. Esta clase implementa la interfaz `IAddressValidator` y se encarga de traducir las solicitudes y respuestas entre el modelo utilizado por la aplicación y el formato requerido por el servicio externo.

A continuación se muestran los tres fragmentos de código referidos en la solución.

```
public interface IAddressValidator
{
    Task<bool> IsValidAsync(string address);
}
```

Código 1. Puerto en el núcleo: `IAddressValidator`.

```
public class OrderProcessor
{
    private readonly IAddressValidator _addressValidator;

    public OrderProcessor(IAddressValidator addressValidator)
    {
        _addressValidator = addressValidator;
    }
}
```

Código 2. `OrderProcessor` recibe el puerto por inyección de dependencias.

```
public class HttpAddressValidatorAdapter : IAddressValidator
{
    private readonly HttpClient _httpClient;
```

```
    public HttpAddressValidatorAdapter(HttpClient httpClient)
    {
        _httpClient = httpClient;
    }
}
```

Código 3. Adaptador en infraestructura: `HttpAddressValidatorAdapter`.

De esta forma, las dependencias del código fuente apuntan hacia la abstracción situada en el núcleo. `OrderProcessor` depende de `IAddressValidator`, mientras que `HttpAddressValidatorAdapter` implementa esa misma interfaz. La aplicación no necesita conocer la clase concreta que realiza la comunicación externa.

Este diseño es compatible con la Arquitectura Limpia, la Arquitectura Hexagonal y la Arquitectura Cebolla. En la primera, respeta la Regla de Dependencia; en la segunda, la interfaz funciona como un puerto y la implementación como un adaptador; y en la tercera, mantiene la infraestructura en las capas externas, dependiente de las abstracciones internas.

4.3. Impacto en la testabilidad, la mantenibilidad y la flexibilidad

La introducción de la interfaz `IAddressValidator` aporta tres beneficios fundamentales:

- **Testabilidad:** `OrderProcessor` puede probarse de forma unitaria sin realizar llamadas HTTP reales. Para ello, se utiliza una implementación simulada (*mock* o *stub*) de `IAddressValidator`, que devuelve resultados controlados durante las pruebas. Así se comprueba el comportamiento de la lógica de negocio de manera rápida, aislada y reproducible, sin depender de la disponibilidad del servicio externo ni de la biblioteca HTTP utilizada.
- **Mantenibilidad:** al separar las responsabilidades, los cambios en la biblioteca HTTP o en los mecanismos de comunicación externa quedan concentrados principalmente en el adaptador. Esto reduce el impacto de las modificaciones tecnológicas sobre la lógica de negocio y facilita el mantenimiento del sistema.
- **Flexibilidad y evolución:** si en el futuro se sustituye el proveedor de validación de direcciones, puede desarrollarse un nuevo adaptador que implemente `IAddressValidator`, manteniendo intacta la lógica de `OrderProcessor`, siempre que el contrato y el comportamiento requerido por la aplicación continúen siendo compatibles. Si el nuevo proveedor utiliza formatos de datos diferentes, el adaptador se encarga de realizar las conversiones necesarias.

En conclusión, aplicar el Principio de Inversión de Dependencias permite desacoplar la lógica de negocio de los detalles tecnológicos, favoreciendo las pruebas unitarias, la mantenibilidad y la evolución del sistema. La clave del diseño consiste en que el núcleo defina las abstracciones que necesita y que la infraestructura implemente dichas abstracciones, en lugar de imponer sus detalles al núcleo de la aplicación.

4.4. Pruebas unitarias de la solución propuesta

Para comprobar el funcionamiento de la solución basada en el Principio de Inversión de Dependencias (DIP), se implementaron pruebas unitarias utilizando xUnit en .NET.

El objetivo es verificar el comportamiento de la clase `OrderProcessor` ante diferentes resultados de la validación de una dirección, sin depender de un servicio externo real.

Para ello, se creó la clase `FakeAddressValidator`, una implementación simulada de la interfaz `IAddressValidator`. Esta devuelve un resultado controlado por la prueba, lo que permite comprobar la lógica de negocio de forma aislada, sin realizar peticiones HTTP ni depender de la disponibilidad de una API externa.

Se definieron dos casos de prueba:

- **Dirección válida:** se configura el validador simulado para que devuelva `true`. Se comprueba que el método `ProcessOrderAsync` finalice sin lanzar una excepción cuando el validador considera válida la dirección.
- **Dirección inválida:** se configura el validador para que devuelva `false`. Se verifica que `OrderProcessor` lance una excepción de tipo `ArgumentException`, rechazando la dirección.

La ejecución satisfactoria de ambas pruebas permite verificar que el procesador responde correctamente a los resultados de la validación. Además, ilustra una ventaja importante del DIP: la lógica de negocio depende de la abstracción `IAddressValidator`, y no de una implementación concreta que se comunique directamente con un servidor externo. Por tanto, es posible sustituir el adaptador HTTP por una implementación simulada durante las pruebas sin modificar la clase `OrderProcessor`.

Estas pruebas verifican el comportamiento del procesador y su capacidad para trabajar con diferentes implementaciones de la abstracción. No verifican la comunicación real con el servicio externo ni la persistencia o finalización completa de un pedido, funcionalidades que quedan fuera del alcance de este ejemplo.

Pregunta 5. Análisis de un Problema Real: El Kitty Problem y el Principio de Abierto/Cerrado

Aclaración: en el libro que indica el enunciado (*Fundamentals of Software Architecture*, de Richards y Ford) no encontramos que se hablase del *Kitty Problem*. Dicho problema está en el libro de Robert C. Martin, *Clean Architecture*.

5.1. Descripción del problema

Martin (2017, pp. 242-243) retoma un sistema agregador de taxis. El sistema conoce a muchos proveedores de taxis de una ciudad y permite que los clientes pidan viajes, eligiendo según criterios como el tiempo de recogida, el costo, el nivel de lujo y la experiencia del conductor. Para que el sistema fuera escalable se construyó con muchos microservicios pequeños, cada uno a cargo de un equipo reducido. Los servicios eran cuatro:

- ***TaxiUI*:** atiende a los clientes, que piden taxis desde dispositivos móviles.
- ***TaxiFinder*:** revisa los inventarios de los distintos *TaxiSuppliers*, determina qué taxis son candidatos y los deja en un registro de datos de corta vida asociado al usuario.
- ***TaxiSelector*:** toma los criterios del usuario y elige un taxi entre los candidatos.
- ***TaxiDispatcher*:** recibe el taxi elegido y realiza el pedido.

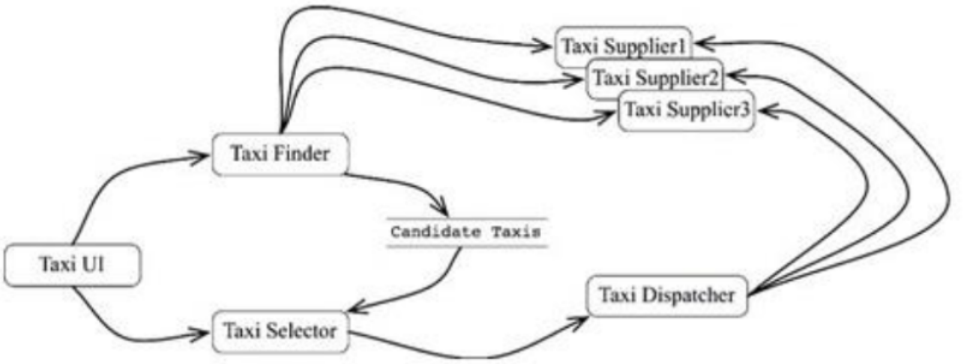


Figure 27.1 Services arranged to implement the taxi aggregator system

Después de más de un año de operación, el área de marketing anuncia un servicio de entrega de gatitos a domicilio. Habría puntos de recolección en la ciudad; un taxi cercano recogería el gatito y lo llevaría a su destino. Solo un proveedor participaría al inicio. Además, los conductores alérgicos a los gatos no deben ser elegidos, y un vehículo que haya transportado gatitos en los últimos tres días no debe asignarse a clientes que declaren alergia (Martin, 2017, p. 243).

Martin se pregunta cuántos servicios deberían cambiar y responde que todos. El desarrollo y el despliegue tendrían que coordinarse con mucho cuidado, lo que significa que los servicios estaban acoplados y no podían desarrollarse, desplegarse ni mantenerse de forma independiente. Este es el problema de las preocupaciones transversales (*cross-cutting concerns*): cualquier sistema las enfrenta, esté o no orientado a servicios, y las descomposiciones funcionales como la de la Figura 27.1 son muy vulnerables a las características nuevas que atraviesan todos los comportamientos (Martin, 2017, p. 243).

5.2. Por qué la división por verbos llevó a esto

Los servicios se definieron a partir de los pasos del proceso (buscar, seleccionar, despachar), es decir, por verbos. Martin muestra que esa estructura se apoya en dos falacias:

- **Falacia del desacoplamiento** (Martin, 2017, pp. 240-241). Los servicios parecen desacoplados porque están en procesos distintos y se comunican por interfaces. Pero siguen acoplados por los datos que comparten. En el ejemplo, *TaxiFinder* deja un registro con los candidatos que luego lee *TaxiSelector*; si cambia un campo de ese registro, todos los servicios que lo usan deben cambiar. Martin añade que una interfaz de servicio no es más rigurosa que la de una función, por lo que esa ventaja es en buena medida ilusoria.
- **Falacia del desarrollo y despliegue independiente** (Martin, 2017, p. 241). Se cree que cada equipo puede desarrollar y desplegar su servicio por separado. Pero en la medida en que los servicios están acoplados por datos o por comportamiento, el desarrollo, el despliegue y la operación deben coordinarse.

Cada servicio mezcla lógica general (buscar, seleccionar) con lógica propia de los viajes. Por eso, cuando aparece otro tipo de pedido, no existe un lugar donde "enchufarlo". Los límites del sistema coinciden con los pasos funcionales y no con los ejes de cambio del negocio. Los límites arquitectónicos no caen entre los servicios; los atraviesan y los dividen en componentes (Martin, 2017, p. 246).

5.3. Cómo diseñarlo para que el cambio afecte a un solo componente

Martin (2017, pp. 244-246) propone aplicar SOLID para que las clases puedan extenderse de forma polimórfica. La lógica general de los servicios originales permanece en las clases base del modelo de objetos; lo específico de los viajes pasa a un componente *Rides*, y la nueva característica se coloca en un componente *Kittens*. Ambos redefinen las clases abstractas del núcleo mediante *Template Method* o *Strategy*, siguen la Regla de Dependencia (Martin, 2017, p. 203) (apuntan hacia el núcleo, que no los conoce) y sus clases las crean fábricas controladas por la UI. Así, al añadir los gatitos solo cambia la *TaxiUI*; el resto se incorpora como un nuevo JAR, Gem o DLL que se carga dinámicamente, y la característica queda desacoplada y puede desarrollarse y desplegarse de forma independiente.

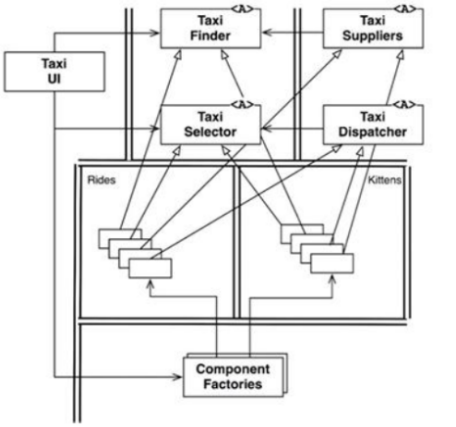


Figure 27.2 Using an object-oriented approach to deal with cross-cutting concerns

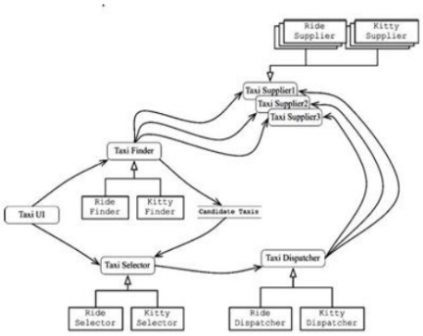


Figure 27.3 Each service has its own internal component design, enabling new features to be added as new derivative classes

```java
//NÚCLEO
public abstract class TaxiSelector {
    public Taxi select(List<Taxi> candidates, Criteria c) {
        List<Taxi> eligible = filter(candidates, c); // paso variable
        return pickBest(eligible, c);                // paso común
    }
    protected abstract List<Taxi> filter(List<Taxi> t, Criteria c);
    protected Taxi pickBest(List<Taxi> t, Criteria c) { /* lógica común */ }
}

//PLUGIN Rides
public class RideSelector extends TaxiSelector {
    protected List<Taxi> filter(List<Taxi> t, Criteria c) {
        /* reglas de viaje */
    }
}

// PLUGIN Kittens
public interface KittenHistory {           // puerto definido en el plugin
    boolean usedForKittensWithin(TaxiId id, int days);
}

public class KittenSelector extends TaxiSelector {
    private final KittenHistory history;
    public KittenSelector(KittenHistory history) { this.history = history; }
```

```
    protected List<Taxi> filter(List<Taxi> t, Criteria c) {
        return t.stream()
            .filter(x -> !x.driver().isAllergicToCats())
            .filter(x -> !c.customerDeclaresAllergy()
                      || !history.usedForKittensWithin(x.id(), 3))
            .toList();
    }
}

// ADAPTADOR
public class SupplierApiKittenHistory implements KittenHistory {
    public boolean usedForKittensWithin(TaxiId id, int days) {
        /* llama a la API del proveedor */
    }
}
public class SelectorFactory {
    TaxiSelector forOrder(OrderType type) {
        return type == OrderType.KITTEN
            ? new KittenSelector(new SupplierApiKittenHistory())
            : new RideSelector();
    }
}
```

Listing 2: Diseño extensible del *Kitty Problem*: núcleo, plugins y adaptador.

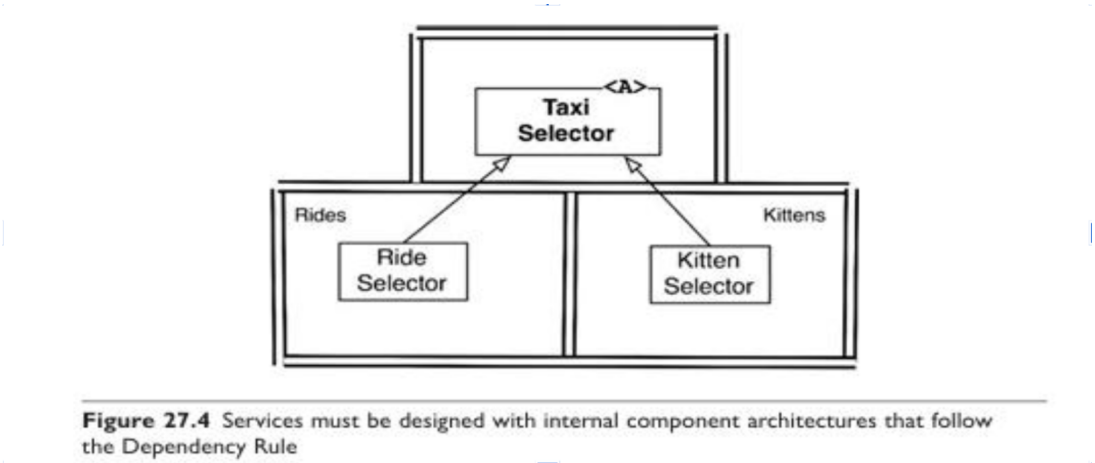


Figure 27.4 Services must be designed with internal component architectures that follow the Dependency Rule

5.4. El Principio de Abierto/Cerrado

El Principio de Abierto/Cerrado establece que un artefacto de software debe estar abierto a la extensión y cerrado a la modificación (Martin, 2017, cap. 8, pp. 69-75).

El sistema queda abierto a la extensión (se agrega *Kittens* como nueva derivada) y cerrado a la modificación (no se edita el código del núcleo ni de *Rides*). Martin lo describe pensando en un servicio como un conjunto de clases abstractas en uno o más JAR, y en cada nueva característica como otro JAR con clases que las extienden. Desplegar la característica deja de ser un redespliegue de servicios y pasa a ser añadir un JAR a la ruta de carga (Martin, 2017, p. 246).

5.5. Papel de los adaptadores en la extensión

- **Adaptadores de salida.** *Kittens* necesita información que el núcleo no debe conocer: la API del proveedor de taxis que participa, los puntos de recolección o un sistema de alergias. Cada dependencia se oculta tras una interfaz y se implementa en un adaptador de la capa exterior, igual que el repositorio que Palermo (2008a, 2008b) define como interfaz en el núcleo e implementa en la infraestructura.
- **Adaptadores de entrada.** Los controladores y presenters de la UI traducen el pedido del cliente ("quiero un gatito") en llamadas al núcleo, a través de la fábrica.
- **Efecto.** Si mañana otro proveedor se suma o se retira, solo se agrega o cambia un adaptador, sin tocar el núcleo ni los plugins. Martin lo plantea en términos equivalentes para la base de datos y la UI, que pueden reemplazarse como plugins (Martin, 2017, p. 171).

Desde la perspectiva de DDD, *Rides* y *Kittens* son variantes del mismo concepto de pedido de transporte, cada una con sus propias reglas, por lo que se modelan como extensiones de un núcleo estable y no como servicios independientes por función. Esto coincide con la Arquitectura Onion, donde el modelo de dominio ocupa el centro, las capas externas dependen de las internas y nunca al revés (Palermo, 2008a). Por eso agregar *Kittens* solo implica añadir un componente exterior que depende del núcleo.

Pregunta 6. Evaluación de la Arquitectura N-Capas frente a Requisitos de Atributos de Calidad

Richards (2015, pp. 8-9) califica la arquitectura en capas con valoración baja en agilidad general, facilidad de despliegue, rendimiento y escalabilidad, y alta en facilidad de desarrollo. Las razones que ofrece son estructurales: la naturaleza monolítica de la mayoría de las implementaciones, el fuerte acoplamiento entre componentes, el hecho de que un cambio pequeño obliga a redesplegar toda la aplicación o una parte grande de ella, y una granularidad de escalado demasiado amplia. Richards y Ford (2020, cap. 10) mantienen ese diagnóstico y añaden que, a medida que la aplicación crece, se deterioran la mantenibilidad, la agilidad, la testabilidad y la desplegabilidad.

Esto es clave para evaluar la propuesta del estudiante: las causas señaladas son el monolito y el acoplamiento, no la longitud del camino que recorre cada petición.

6.1. Análisis de la afirmación: capas abiertas y fast-lane reader

En una capa cerrada, toda petición debe pasar por la capa inmediatamente inferior; en una capa abierta, la petición puede saltársela y llegar a la siguiente (Richards, 2015, pp. 3-5). El *fast-lane reader*, según Richards y Ford (2020, cap. 10), consiste en que la capa de presentación acceda directamente a la base de datos para lecturas simples, sin pasar por las capas intermedias, lo cual exige que las capas de negocio y persistencia estén abiertas.

La afirmación es parcialmente correcta. Esta técnica reduce el procesamiento innecesario: Richards y Ford (2020, cap. 10) explican que el paso de peticiones por capas sin lógica implica instanciación y procesamiento superfluos que afectan la memoria y el rendimiento.

Por tanto, puede mejorar el rendimiento de las lecturas simples. Pero no mitiga los otros tres problemas que menciona el enunciado:

- **Agilidad:** Depende de aislar el cambio. Con el acceso directo, la presentación pasa a depender de la persistencia, de modo que un cambio en ésta afecta también a aquella (Richards, 2015, pp. 3-4). La agilidad empeora.
- **Facilidad de despliegue:** No cambia, porque las capas abiertas no alteran la topología física: el sistema sigue siendo una sola unidad de despliegue, o dos en la variante con presentación separada (Richards y Ford, 2020, cap. 10).
- **Escalabilidad:** Richards (2015, p. 9) la atribuye a la implementación monolítica y a la granularidad demasiado amplia. Saltarse capas no permite escalar funciones por separado; para lograrlo en un monolito se necesitan técnicas complejas como multihilo o mensajería interna, para las que esta arquitectura no está bien preparada (Richards y Ford, 2020, cap. 10).

El concepto de *layers of isolation* establece que los cambios en una capa generalmente no afectan a las demás, siempre que los contratos entre ellas se mantengan. Para que se cumpla, las capas que participan en el flujo principal de la petición deben ser cerradas (Richards, 2015, pp. 3-4; Richards y Ford, 2020, cap. 10). Si la presentación accede directamente a la persistencia, un cambio en el SQL repercute tanto en la capa de negocio como en la de presentación, produciendo, en palabras de Richards (2015, p. 4), una aplicación muy acoplada, difícil y costosa de cambiar.

Esto coincide con otras fuentes. Palermo (2008a) sostiene que el mayor problema de las capas es el acoplamiento de la UI y de la lógica de negocio con el acceso a datos, y recuerda que las dependencias transitivas siguen siendo dependencias. Cockburn (2005) añade que, en los dibujos por capas, la gente no toma en serio las líneas entre ellas y deja que la lógica se filtre por los límites. Abrir las capas formaliza esa filtración.

Conviene matizar que no toda capa abierta es un error. Richards (2015, pp. 4-5) presenta un caso legítimo: una capa de servicios compartidos marcada como abierta para que el negocio pueda llegar a la persistencia sin pasar por ella. Lo problemático es abrir las capas del flujo principal, porque entonces se pierde el aislamiento que justifica el estilo.

6.2. El anti-patrón sinkhole y la regla del 80/20

El *architecture sinkhole* ocurre cuando las peticiones atraviesan varias capas como simple paso de datos, sin que ninguna aplique lógica (Richards, 2015, p. 7; Richards y Ford, 2020, cap. 10). Toda arquitectura en capas tiene algunos casos así. La regla del 80/20 dice que es aceptable que alrededor del 20 % de las peticiones sean de este tipo y que el 80 % tenga lógica asociada. Si la proporción se invierte y la mayoría son de paso, el estilo en capas probablemente no es el adecuado para el problema (Richards y Ford, 2020, cap. 10).

Es importante precisar la relación entre este anti-patrón y las capas abiertas. Según las fuentes, el *sinkhole* no es una consecuencia de abrir capas sino lo que motiva abrirlas: Richards (2015, pp. 7-8) sugiere considerar capas abiertas cuando la proporción se invierte, pero advierte que será más difícil controlar el cambio por la falta de aislamiento. El problema es que la estructura por roles técnicos obliga a toda petición a recorrer todas las capas, aunque no haya nada que hacer en ellas. Abrir las capas elimina el recorrido inútil, pero a cambio de renunciar al aislamiento; es decir, intercambia un problema por otro sin salir del estilo. Si, además, el 80 % de las peticiones son de paso, la propia regla indica que lo que falla es la elección del estilo.

6.3. Por qué Hexagonal y Limpia son más adecuadas para alta escalabilidad y agilidad

La diferencia de fondo está en cómo tratan las dependencias. En N-Capas el negocio queda acoplado a la infraestructura (Palermo, 2008c). En Hexagonal y Limpia, el núcleo no depende de nada exterior: Martin (2017, pp. 202-203) resume que estas arquitecturas producen sistemas independientes de *frameworks*, de la UI y de la base de datos, y testables sin ninguno de ellos, y formula la Regla de Dependencia según la cual el código fuente solo puede apuntar hacia adentro, hacia las políticas de mayor nivel. De ahí se derivan varias ventajas:

- **Agilidad:** Los detalles cambiantes quedan en la capa exterior. Palermo (2008c) afirma que se puede cambiar el código de cualquier capa externa sin afectar al núcleo de la aplicación. Cockburn (2005) explica que, si la base de datos cambia de SQL a otro tipo, el diálogo a través del puerto no tiene por qué cambiar.
- **Testabilidad:** El núcleo puede ejecutarse con adaptadores de prueba, por ejemplo una base de datos en memoria (Cockburn, 2005).

El *fast-lane reader* se resuelve sin romper el aislamiento. Cockburn (2005) propone definir las interfaces por propósito y no por tecnología, con varios adaptadores por puerto. La necesidad de lecturas rápidas se cubre con un puerto de consulta definido en el núcleo, cuyo adaptador puede leer de SQL, de una réplica o de una caché. Palermo (2008c) señala además que en la *Onion* cualquier capa externa puede llamar directamente a cualquier capa interna, porque las dependencias siempre apuntan hacia el centro. La diferencia con el *fast-lane reader* es que allí el salto va hacia la infraestructura concreta, y aquí hacia una abstracción del núcleo.

Escalabilidad: una precisión necesaria. Hexagonal y Limpia no escalan por sí mismas, porque describen la dirección de las dependencias y no la forma de despliegue; un sistema Limpio sigue pudiendo ser un único monolito. Lo que aportan es que dejan abiertas las opciones. Martin (2017, pp. 149 y 155-157) explica que una buena arquitectura permite que un sistema nazca como monolito, evolucione hacia unidades desplegables de forma independiente y llegue a servicios, y que también pueda revertir ese camino, precisamente porque aísla sus componentes y no asume el medio de comunicación entre ellos. En N-Capas, en cambio, el dominio queda disperso por todas las capas, lo que dificulta aplicar cambios sobre él y hace que el diseño guiado por el dominio funcione peor (Richards y Ford, 2020, cap. 10). Al combinar el núcleo desacoplado con una partición por dominio, es posible extraer y escalar componentes de forma independiente.

Debe reconocerse también el compromiso (*trade-off*). Richards y Ford (2020, cap. 10) indican que la arquitectura en capas es una buena elección, sobre todo como punto de partida, para aplicaciones pequeñas y simples o con presupuestos y plazos muy ajustados. Palermo (2008a) afirma por su parte que la *Onion* no es apropiada para sitios web pequeños, sino para aplicaciones empresariales de larga vida y comportamiento complejo. La ventaja de Hexagonal y Limpia aparece cuando crecen la complejidad, el tiempo de vida y la presión por cambiar.

Conclusiones

El análisis desarrollado permite concluir que la diferencia entre las arquitecturas estudiadas reside en la forma en la que sus representaciones dirigen la atención hacia una manera

de organizar responsabilidades y controlar dependencias. La arquitectura Hexagonal hace explícita la distinción entre la aplicación y su entorno, mostrando que la interfaz de usuario, la persistencia y otros servicios son elementos externos que deben relacionarse con el núcleo mediante límites definidos (Cockburn, 2005). También se precisó que Onion junto a la arquitectura Hexagonal comparten el propósito de aislar la infraestructura, aunque destacan aspectos complementarios. Frente a ellas, la organización tradicional por capas permite distinguir responsabilidades, pero esa separación no garantiza por sí sola la independencia del negocio cuando las capas superiores dependen, directa o transitivamente, de implementaciones de acceso a datos (Palermo, 2008).

La solución práctica traslada esta discusión al diseño de componentes. Al utilizar IAddressValidator, OrderProcessor puede expresar su necesidad de validar una dirección sin incorporar directamente los detalles del mecanismo HTTP que realiza esa tarea. La implementación externa queda separada del comportamiento del procesador, y puede sustituirse durante las pruebas sin modificar dicha clase. Esta distribución ilustra el principio mencionado: mantener los contratos próximos a las necesidades del núcleo y las implementaciones tecnológicas fuera de él. Las pruebas con xUnit aportan evidencia sobre dos comportamientos concretos: según la ejecución satisfactoria descrita, el procesador no lanza una excepción cuando la validación devuelve true y rechaza la dirección mediante una ArgumentException cuando devuelve false.

En conjunto, el seminario muestra que el valor de una arquitectura no reside únicamente en nombrar capas o reproducir un diagrama, sino en convertir sus límites conceptuales en dependencias concretas que preserven la autonomía del negocio. La comparación arquitectónica, la solución mediante abstracciones y las pruebas aisladas permiten llegar a la conclusión que separar las reglas de negocio de los detalles externos permite comprender mejor el sistema y comprobar sus comportamientos sin exigir que toda la infraestructura esté disponible.

Bibliografía

- **Cockburn, A. (2005, 4 de septiembre).** *The Hexagonal (Ports & Adapters) Architecture*. Consultado en: https://alistair.cockburn.us/hexagonal-architecture/

- **De la Torre Llorente, C., Zorrilla Castro, U., Ramos Barroso, M. A., & Calvarro Nelson, J. (2010).** *Guía de Arquitectura N-Capas orientada al Dominio con .NET 4.0 (Borrador)*. Microsoft Ibérica.

- **Evans, E. (2003).** *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley.

- **Fowler, M. (2002).** *Patterns of Enterprise Application Architecture*. Addison-Wesley.

- **Martin, R. C. (2003).** *Agile Software Development: Principles, Patterns, and Practices*. Prentice Hall.

- **Martin, R. C. (2012).** *The Clean Architecture*. Clean Coder Blog. Consultado en: https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html

- **Martin, R. C. (2017).** *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall. Consultado el 15 de octubre de 2026.

- **Palermo, J. (2008a, 29 de julio).** *The Onion Architecture: part 1 | Programming with Palermo*. Jeffrey Palermo. Consultado el 15 de octubre de 2026, en: https://jeffreypalermo.com/2008/07/the-onion-architecture-part-1/

- **Richards, M., & Ford, N. (2020).** *Fundamentals of Software Architecture: An Engineering Approach*. O'Reilly Media.

- **Yaghini, V. (2025, 23 de enero).** *Organize your application code in three-tier architecture*. Consultado el 16 de octubre de 2026, en: https://openclassrooms.com/en/courses/5684146-create-web-applications-efficiently-with-the-spring-boot-mvc-fram 6156961-organize-your-application-code-in-three-tier-architecture

Declaración de Uso de la Inteligencia Artificial

Para la revisión y el desarrollo de este seminario, se utilizaron como herramientas de apoyo los modelos de inteligencia artificial ChatGPT (GPT-6) y Gemini (flash 3), este último empleado específicamente para la elaboración y corrección de los elementos en LaTeX. No obstante, el equipo de trabajo asume la responsabilidad sobre la veracidad de la información, la adecuación de las referencias y la integridad del contenido final presentado.